

Deep Learning và CNN — Cơ sở lý thuyết

Assignment 4 · Deep Learning · CNN · Improved CNN Models

Tài liệu trình bày cơ sở lý thuyết của bài: deep learning hoạt động thế nào, CNN gồm những thành phần gì, và các kỹ thuật cải tiến CNN đem lại bao nhiêu.

Mọi con số lấy trực tiếp từ `results/all_runs.csv` và `results/04_variants.json` của bài này — không phải số ví dụ.

1. Deep Learning là gì?

1.1 Định nghĩa ngắn

Deep Learning là nhánh của Machine Learning dùng mạng nơ-ron **hiều tầng** (deep = sâu = nhiều tầng) để học biểu diễn dữ liệu theo **hiều mức trừu tượng**, tầng sau xây trên đặc trưng tầng trước.

Điểm khác biệt cốt lõi so với ML truyền thống:

ML truyền thống:

Dữ liệu → CON NGƯỜI thiết kế đặc trưng → Thuật toán học → Kết quả
 ↑
 tốn công, cần chuyên gia miền

Deep Learning:

Dữ liệu → MÁY tự học đặc trưng + tự học phân loại → Kết quả
|———— học đồng thời, end-to-end ————|

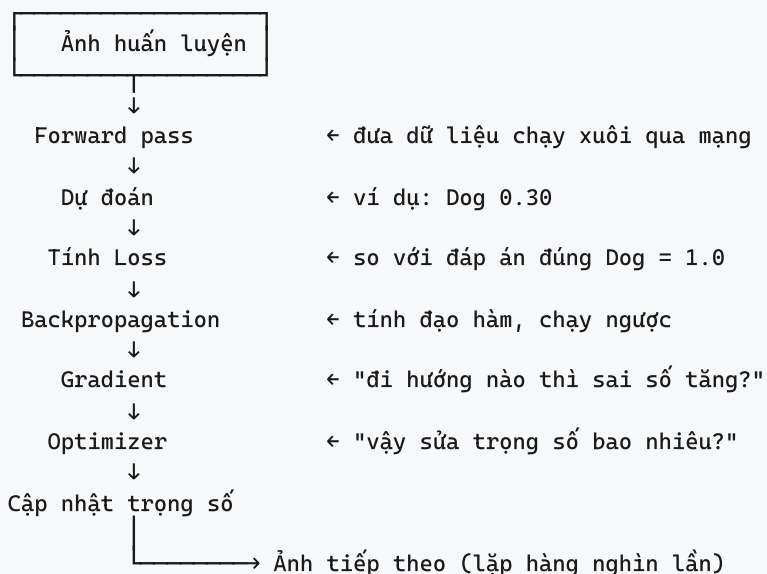
Trong bài này điều đó thể hiện rõ: **không ai viết code "tìm cạnh" hay "tìm bánh xe"**. Ta chỉ đưa ảnh CIFAR-10 thô vào, mạng tự học ra các bộ lọc hữu ích.

1.2 Bốn thành phần bắt buộc

Bất kỳ mô hình deep learning nào cũng gồm:

Thành phần	Vai trò	Trong bài này
Kiến trúc	Định nghĩa phép biến đổi dữ liệu	MLP (tabular), CNN (ảnh)
Loss function	Đo mức độ sai	Cross-entropy
Optimizer	Quyết định sửa trọng số thế nào	SGD / Adam
Dữ liệu	Nguồn để học	3 datasets

1.3 Mạng học bằng cách nào — vòng lặp huấn luyện



1.4 Gradient — giải thích cho người không học toán

Tưởng tượng bạn là **quả bóng đứng trên sườn đồi**, mắt bị bịt. Bạn muốn xuống đáy đồi nhưng không nhìn được toàn cảnh. Bạn chỉ cảm nhận được: "*bước một bước nhỏ về phía này thì lên hay xuống?*"

Đó chính là **gradient**.

Trong ví dụ	Trong mạng nơ-ron
Quả bóng	Bộ trọng số của mạng
Độ cao của đồi	Loss (sai số)
Đi xuống dốc	Giảm sai số
Độ dài mỗi bước	Learning rate

Công thức cập nhật:

```
w_mới = w_cũ - η × (∂L/∂w)

w      = trọng số
L      = loss
∂L/∂w = gradient
η      = learning rate (độ dài bước)
```

Ví dụ số: $w = 0.50$, $\text{gradient} = 0.20$, $\eta = 0.01 \rightarrow w_{\text{mới}} = 0.50 - 0.01 \times 0.20 = 0.498$

Learning rate chọn sai thì sao:

- Quá nhỏ (0.00001) → đi từng bước tí hon, huấn luyện lâu vô tận.
- Quá lớn → nhảy vọt qua đáy, loss dao động hoặc tệ đi.
- Vừa đủ → giảm nhanh lúc đầu, tinh chỉnh dần về sau.

1.5 Backpropagation — đừng nhầm với nguyên hàm

Câu hỏi hay gặp: "*Backpropagation có giống nguyên hàm/tích phân không?*" **Không**. Nó gần với **đạo hàm** hơn nhiều.

Vì w_1 nằm xa loss, ta dùng **chain rule** (quy tắc chuỗi):

$x \xrightarrow{w_1} y \xrightarrow{w_2} z \rightarrow L$

$$\partial L / \partial w_1 = (\partial L / \partial z) \times (\partial z / \partial y) \times (\partial y / \partial w_1)$$

Thông tin sai số lan **ngược** từ cuối về đầu mạng — nên gọi là *backpropagation*.

Phân biệt hai khái niệm hay bị gộp:

	Trả lời câu hỏi	Việc cụ thể
Backpropagation	"Gradient của loss theo từng tham số là bao nhiêu?"	Tính gradient
Gradient Descent / Adam	"Có gradient rồi thì sửa thế nào?"	Dùng gradient để cập nhật

Trong `scratch_nn.py` của bài, toàn bộ chuỗi đạo hàm này được viết tay bằng NumPy — đây là phần thể hiện rõ nhất cơ chế học của mạng.

2. CNN understanding

2.1 Vì sao ảnh cần CNN chứ không phải MLP thường

Ảnh CIFAR-10 kích thước $32 \times 32 \times 3 = 3072$ giá trị. Nếu nối thẳng vào một lớp Dense 1024 nơ-ron:

$3072 \times 1024 \approx 3.1$ triệu tham số – chỉ riêng lớp đầu tiên

Ba vấn đề:

1. **Quá nhiều tham số** → cần cực nhiều dữ liệu, dễ overfit.
2. **Mất cấu trúc không gian** → flatten làm hai pixel cạnh nhau trở nên không liên quan.
3. **Không bất biến vị trí** → con mèo lệch sang phải 5 pixel thành mẫu hoàn toàn khác.

CNN giải quyết cả ba bằng **chia sẻ trọng số** (weight sharing): một kernel nhỏ quét khắp ảnh, nên học "cạnh dọc" một lần là dùng được ở mọi vị trí.

Đối chiếu số thật trong bài: SmallCNN cho CIFAR-10 chỉ có **545,098 tham số**, còn M1 tối giản chỉ **72,730 tham số** — ít hơn hàng chục lần so với MLP nối thẳng, mà vẫn đạt ~69–72% accuracy.

2.2 Convolution — trái tim của CNN

Kernel là ma trận nhỏ chứa **trọng số học được**, trượt qua ảnh, mỗi vị trí làm phép nhân từng phần tử rồi cộng lại.

Vùng ảnh	Kernel
1 2	1 0
4 5	0 -1

×

$= 1 \times 1 + 2 \times 0 + 4 \times 0 + 5 \times (-1) = -4$

Kernel trượt hết ảnh → thu được **Feature Map** (bản đồ đặc trưng).

Điểm quan trọng nhất của convolution: lập trình viên **không** tự định nghĩa kernel nào dò cạnh, kernel nào dò góc. Các giá trị $w_1 \dots w_9$ là **tham số học được**, mạng tự tìm ra qua backpropagation.

2.3 Stride và Padding

Stride — kernel nhảy bao nhiêu pixel mỗi bước:

stride = 1 → trượt từng pixel, giữ gần như mọi vị trí
stride = 2 → nhảy 2 pixel, feature map nhỏ đi một nửa
⇒ stride càng lớn, feature map càng nhỏ

Padding — chèn thêm viền (thường là số 0) quanh ảnh:

Không padding: 5×5, kernel 3×3 → output 3×3 (teo dần)
Có padding=1: 5×5, kernel 3×3 → output 5×5 (giữ nguyên)

VALID = không padding
SAME = padding vừa đủ để giữ kích thước

Padding có ích vì: giữ kích thước không gian, và cho **pixel ở rìa** được xử lý nhiều lần hơn thay vì bị bỏ quên.

2.4 ReLU — vì sao bắt buộc phải có

$\text{ReLU}(x) = \max(0, x)$

Đầu vào: -5 2 -3 Sau ReLU: 0 2 0
 4 -1 7 4 0 7

Lý do tồn tại: **đưa tính phi tuyến vào mạng**. Nếu không có hàm kích hoạt, xếp 50 tầng tuyến tính lên nhau vẫn chỉ tương đương **một** phép biến đổi tuyến tính — mạng sâu trở nên vô nghĩa.

ReLU phổ biến vì đơn giản, tính nhanh, và hiệu quả trong mạng sâu.

2.5 Pooling

Max Pooling 2×2 lấy giá trị lớn nhất trong mỗi ô vuông:

Feature Map		Kết quả
1 3 2 4	→	
5 6 1 2		6 4
7 2 9 3		7 9
4 1 5 8		

Ba lợi ích: **giảm tính toán**, **giảm bộ nhớ**, và **chịu được dịch chuyển nhỏ** (cạnh xê dịch vài pixel, mạng vẫn nhận ra đặc trưng đó).

2.6 Flatten → Fully Connected → Softmax

Feature maps 7 × 7 × 64
↓ Flatten
3136 giá trị (vì 7 × 7 × 64 = 3136)
↓ Fully Connected
Kết hợp các đặc trưng
↓ Softmax
Cat = 0.10 | Dog = 0.85 | Horse = 0.05 (tổng = 1.0)

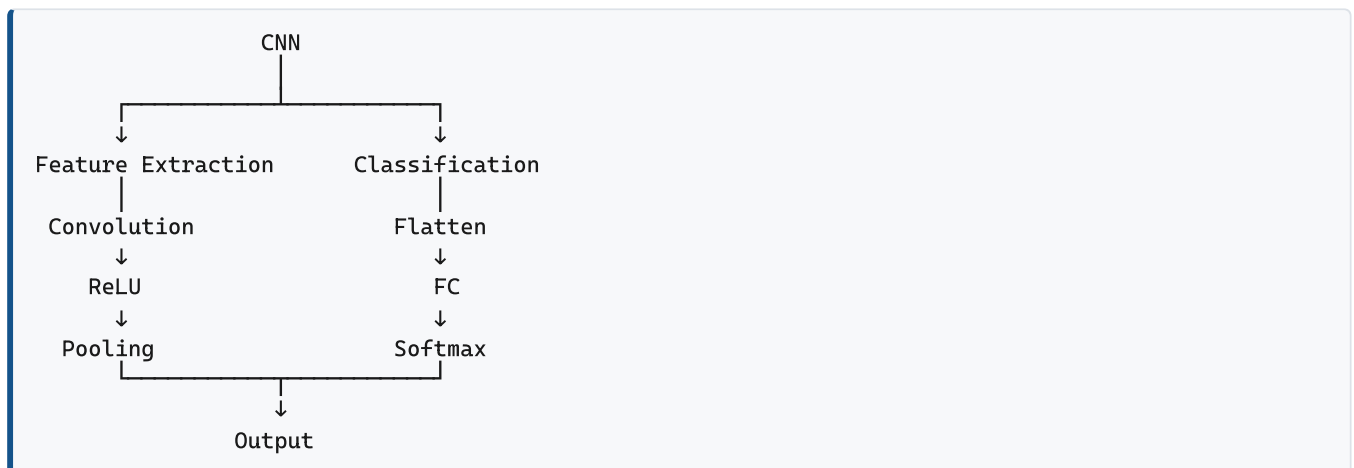
Softmax biến điểm số thô thành **xác suất**:

Điểm thô:	Cat 2.0	Dog 4.0	Horse 1.0	
Softmax:	Cat 0.114	Dog 0.844	Horse 0.042	→ Dự đoán: Dog

2.7 Bảng tổng hợp chức năng các thành phần

Thành phần	Chức năng chính
Input	Nhận ảnh dạng ma trận số
Convolution	Trích xuất đặc trưng cục bộ
Kernel / Filter	Dò mẫu (cạnh, góc, kết cấu)
ReLU	Thêm tính phi tuyến
Pooling	Giảm kích thước không gian
Flatten	Chuyển feature map thành vector
Fully Connected	Kết hợp đặc trưng đã học
Softmax	Chuyển điểm số thành xác suất
Output	Đưa ra dự đoán cuối

2.8 Ý tưởng lớn nhất về CNN — phân cấp đặc trưng



Và theo mức trừu tượng:

Pixel → Cạnh → Kết cấu → Hình dạng → Bộ phận vật thể → Vật thể → Dự đoán

Một câu tóm tắt CNN: CNN học **những mẫu thị giác hữu ích nằm ở đâu** và **các mẫu đó ghép lại thành khái niệm cấp cao thế nào**, rồi dùng chúng để dự đoán.

3. Improved CNN Models

3.1 CNN cơ bản yếu ở chỗ nào

CNN tối giản Conv → ReLU → Pool → Conv → ReLU → Pool → Flatten → Dense :

- Dễ **overfit** — thuộc lòng ảnh huấn luyện thay vì học quy luật.
- Huấn luyện **chậm hoặc không ổn định**.
- **Ngốn dữ liệu**.
- Kiến trúc nông → không trích được đặc trưng đủ phức tạp.

Mỗi kỹ thuật cải tiến dưới đây nhằm vào **một** điểm yếu cụ thể.

3.2 Các kỹ thuật cải tiến

a) Data Augmentation — chống overfit bằng cách tạo thêm dữ liệu

Thay vì chỉ huấn luyện trên một ảnh mèo, sinh ra nhiều biến thể:

Ảnh gốc

- Xoay (rotation)
- Phóng to/thu nhỏ (zoom)
- Dịch ngang / dịch dọc (shift)
- Kéo xiên (shear)
- Lật ngang (horizontal flip)

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2),
])
```

Hai điểm dễ nhầm:

1. **Nhãn không đổi** — ảnh mèo xoay 20° vẫn là mèo.
2. Chỉ augment **tập train**, tuyệt đối không augment tập validation/test — vì test phải phản ánh phân phối dữ liệu thật thì mới đo được khả năng tổng quát hóa.

b) Batch Normalization — ổn định huấn luyện

```
x = Conv2D(32, 3)(x)
x = BatchNormalization()(x)
x = ReLU()(x)
```

Chuẩn hóa giá trị chạy qua mạng về khoảng dễ kiểm soát. Cách nhớ: *"giữ cho các con số đi qua mạng luôn ngoan"*. Kết quả: huấn luyện ổn định hơn và thường nhanh hội tụ hơn.

c) Dropout — chống phụ thuộc vào vài nơ-ron

Bình thường: ● ● ● ● ● ● ●
Có Dropout: ● ○ ● ○ ● ● ○ (○ = tạm tắt trong lượt huấn luyện này)

Tắt ngẫu nhiên một phần nơ-ron khi train → mạng không thể dựa dẫm vào một vài nơ-ron cụ thể → **giảm overfit**.

d) Mạng sâu hơn

Cơ bản: Conv → Pool → Conv → Pool
Mạnh hơn: Conv → Conv → Pool
Conv → Conv → Pool
Conv → Conv → Pool

Lý do: mỗi tầng học một mức trừu tượng cao hơn (cạnh → kết cấu → hình dạng → bộ phận → vật thể).

e) Optimizer tốt hơn và Learning-rate schedule

Thay Gradient Descent cơ bản bằng **Adam**, và hạ dần learning rate:

0.001 → 0.0005 → 0.0001
(bước dài lúc đầu, bước ngắn để tinh chỉnh về sau)

f) Hai nghĩa của "Improved CNN"

Cần phân biệt rõ hai cách hiểu:

Cách	Nội dung
A. Cải tiến CNN cơ bản	Thêm augmentation, BatchNorm, Dropout, optimizer tốt hơn, LR schedule
B. Dùng kiến trúc tiên tiến	VGG, ResNet , DenseNet, EfficientNet, MobileNet

Bài này đi theo **cả hai**: M2 thuộc nhóm A, M3 (residual) và M4 (attention) mượn ý tưởng từ nhóm B.

3.3 Kết quả thực nghiệm — bốn biến thể trên CIFAR-10 đầy đủ 50k

Tất cả cùng PyTorch, cùng 10 epochs, cùng dữ liệu. Chỉ khác kiến trúc.

Mô hình	Tham số	Test accuracy	F1-macro	Train loss	Thời gian (s)	So với M1
M1 Conv+ReLU+Pool	72,730	69.47%	0.6956	0.8397	552.3	—
M2 + BatchNorm	72,954	75.24%	0.7573	0.4489	573.5	+5.77
M3 + Residual	75,786	76.39%	0.7696	0.4485	681.6	+6.92
M4 + Attention (SE)	78,614	73.67%	0.7468	0.4402	766.3	+4.20

Đọc bảng này thế nào

1. BatchNorm là cải tiến đáng giá nhất. Chỉ thêm **224 tham số** (0.3%) nhưng accuracy tăng **5.77 điểm**. Train loss rơi từ 0.84 xuống 0.45 — mạng học nhanh hơn hẳn trong cùng 10 epochs. Đây là ví dụ sách giáo khoa về "cải tiến rẻ mà hiệu quả".

2. Residual cho thêm 1.15 điểm, và chi phí bắt đầu lộ ra. M3 đạt cao nhất (76.39%) nhưng tốn thêm ~19% thời gian huấn luyện so với M2.

3. M4 tụt 2.72 điểm so với M3 — và đây là phần đáng nói nhất.

Điểm mấu chốt: **M4 có train loss thấp nhất bảng (0.4402) nhưng test accuracy lại thấp hơn M3.**

```
M3: train loss 0.4485 → test 76.39%
M4: train loss 0.4402 → test 73.67%
    ↑ khớp train tốt hơn      ↑ nhưng tổng quát hóa kém hơn
```

Đó là dấu hiệu kinh điển của **overfitting**: mô hình khớp dữ liệu huấn luyện tốt hơn nhưng khái quát kém hơn. Giải thích hợp lý:

- Khối SE (Squeeze-and-Excitation) thêm dung lượng mô hình nhưng **10 epochs không đủ** để phần attention học ra trọng số kênh có ý nghĩa.
- Không có regularization bù lại (dropout / augmentation mạnh hơn).
- SE vốn được thiết kế cho mạng sâu như ResNet-50; gắn vào mạng nông ~78k tham số thì lợi ích chưa kịp thể hiện trong khi chi phí thì có ngay.

Kết luận: *thêm kỹ thuật phức tạp không đảm bảo tốt hơn.* Mỗi cải tiến phải được **đo**, không phải được **giả định**. Đây là phát hiện đáng chú ý nhất của bài: mô hình phức tạp nhất không phải mô hình tốt nhất.

3.4 Bảng chứng phụ: dữ liệu quan trọng ngang kiến trúc

Cùng một mô hình SmallCNN, chỉ đổi lượng dữ liệu:

Dataset	PyTorch	TensorFlow/Keras
CIFAR-10 (5k subset)	53.10%	54.85%
CIFAR-10 (full 50k)	71.90%	70.43%

Dữ liệu gấp 10 lần → accuracy tăng khoảng **+17 điểm**, kiến trúc không đổi một dòng.

Câu chốt: *"Trước khi tinh chỉnh kiến trúc, hãy hỏi mình đã có đủ dữ liệu chưa."*

4. Đọc kết quả

4.1 Ba con số đáng chú ý

MNIST - scratch NumPy 98.10% | PyTorch 98.35% → chênh 0.25 điểm
CIFAR-10 - M1 69.47% → M3 76.39% → +6.92 điểm nhờ cải tiến
Tốc độ - scratch 75.14s vs PyTorch 3.69s → nhanh hơn ~20 lần

Con số thứ ba dẫn thẳng vào một ý hay: **cùng thuật toán, khác cách hiện thực**. PyTorch/TensorFlow nhanh hơn nhờ phép toán đã tối ưu ở tầng C/CUDA và tận dụng song song, chứ không phải vì "thuật toán thông minh hơn".

4.2 Cạm bẫy đo lường — dataset Diabetes

Framework	Accuracy	F1-macro
Scratch (NumPy)	60.94%	0.3450
TensorFlow/Keras	60.77%	0.3686
PyTorch	61.05%	0.3694

Accuracy ~61% nhưng **F1-macro chỉ ~0.35**. Khoảng cách lớn này tố cáo **mất cân bằng lớp**: mô hình đoán tốt lớp đa số và gần như bỏ qua lớp thiểu số.

Nếu chỉ báo cáo accuracy thì nghe ổn, nhưng đó là **báo cáo sai bản chất**. Với dữ liệu lệch lớp, accuracy không phải chỉ số đáng tin. Hướng khắc phục: class weighting, oversampling, hoặc đổi sang chỉ số PR-AUC.

5. Câu hỏi thường gặp

Hỏi: Tại sao dùng ReLU mà không dùng sigmoid? Sigmoid bão hòa ở hai đầu, gradient tiến về 0 khiến mạng sâu học rất chậm (vanishing gradient). ReLU giữ gradient bằng 1 ở miền dương, tính cũng rẻ hơn.

Hỏi: Pooling làm mất thông tin, sao vẫn dùng? Đúng là mất, nhưng đổi lại được giảm tính toán và **bất biến dịch chuyển cục bộ**. Thứ bị vứt phần lớn là nhiễu vị trí chính xác — thứ ta không cần để phân loại.

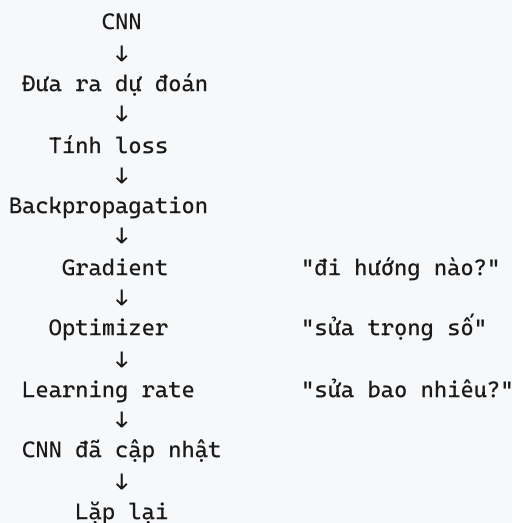
Hỏi: Backpropagation khác gradient descent chỗ nào? Backpropagation **tính** gradient (dùng chain rule). Gradient descent **dùng** gradient đó để cập nhật trọng số. Một cái trả lời "bao nhiêu", một cái trả lời "rồi sao".

Hỏi: Sao M4 phức tạp hơn mà lại kém hơn M3? Train loss của M4 thấp nhất nhưng test accuracy lại thấp hơn — dấu hiệu overfit. Khối SE thêm dung lượng nhưng 10 epochs chưa đủ để nó học có ích, và không có regularization bù lại. Xem mục 3.3.

Hỏi: Sao không dùng ResNet-50 pretrained cho nhanh? Đề bài yêu cầu hiện thực CNN **từ đầu** trên ba framework. Pretrained sẽ cho accuracy cao hơn nhưng không thể hiện được cơ chế bên trong.

Hỏi: Ba framework cho kết quả lệch nhau, có phải code sai? Không. Khởi tạo trọng số ngẫu nhiên, thứ tự shuffle và chi tiết hiện thực khác nhau. Chênh lệch trên MNIST chỉ 0.25 điểm — nằm trong dao động ngẫu nhiên bình thường.

Phụ lục A — bức tranh tổng thể



Và **Data Augmentation, BatchNorm, Dropout, Residual, Attention, LR schedule** là các kỹ thuật ta gắn thêm để vòng lặp này chạy tốt hơn.

Phụ lục B — dựng lại tài liệu

```
# Huấn luyện trước các biến thể (có cache, ngắt giữa chừng chạy lại vẫn tiếp tục)
python prefill_variants.py

# Dựng lại README và toàn bộ PDF notebook
python make_report.py

# Dựng lại riêng tài liệu lý thuyết này
python make_theory_pdf.py
```

Huấn luyện M1–M4 mất tổng cộng khoảng **2,573 giây (~43 phút)**; kết quả đã cache sẵn trong `results/variant_cache/` nên không cần chạy lại.